

PRÁCTICA COMPILADORES

- >_ Alejandro Masmiquel Casado
- >_ Iván Moreno de la Bandera
- >_ Sergio Ruiz Cejudo

Características del lenguaje diseñado

Estructura

- La estructura del lenguaje se basa en la **declaración** de variables y funciones.
- Las **sentencias** (instrucciones a ejecutar), siempre van acabadas en punto y coma (;).
- El punto de entrada principal será un método llamado **main**.

```
vacio main() {  
    cadena nombre;  
    escribir("Hola, soy: ");  
    escribir(nombre);  
}
```

Características del lenguaje diseñado

Declaración y ámbito de variables

Para definir la estructura de los datos, seguimos este orden:

- **Variables:** (*constante*) tipo nombre
 - *Tipos admitidos:* Entero, carácter, booleano o cadena.
- **Arrays:** (*constante*) tipo[tamaño] ([tamaño] [tamaño]) nombre
 - Se añade el tamaño entre corchetes al final del nombre. Los arrays pueden ser de 1, 2 o 3 dimensiones.

El prefijo **constante** es opcional y se usa para asignar valores que no se pueden cambiar.

- **Ámbito Local:** Variables declaradas dentro de funciones.
 - Su uso está restringido exclusivamente a esa función.
- **Ámbito Global:** Variables declaradas fuera de las funciones.
 - Son compartidas y accesibles por todas las funciones del código. Los arrays no pueden ser declarados globalmente.

```
vacio main() {  
    cadena nombre;  
    escribir("Hola, soy: ");  
    escribir(nombre);  
    entero[6] array = [33, 64];  
    entero i = 0;  
    escribir(array[i]);  
    entero[2][3] matriz = [[1, 2, 3], [4, 5, 6]];  
    escribir("Elemento 0,2: ");  
    escribir(matriz[0][2]);  
}
```

Características del lenguaje diseñado

Declaración de funciones

Las funciones son bloques de código diseñados para realizar tareas específicas:

- **Sintaxis General:** tipo nombre(**argumentos*) {bloque de código}
- **Tipos de Retorno:** Determinan qué valor devuelve la función (Entero, carácter, booleano, cadena o array).
- **Funciones sin Retorno:** Se declaran con el tipo vacío cuando no necesitan devolver un resultado.
- **Cuerpo de la Función:** Todo el código debe estar delimitado obligatoriamente por llaves { }.

Una vez definida, la función debe ser llamada para ejecutarse:

- ***Argumentos:** Son los valores o variables que se pasan entre los paréntesis de la función.
- **Llamada a la Función:** Se realiza escribiendo el nombre seguido de los argumentos requeridos: nombre_funcion(arg1, arg2);

```
1 vacio main() {
2     // Llamada a función sin retorno
3     saludar();
4
5     entero num1 = 5;
6     entero num2 = 10;
7
8     // Llamada a función con retorno y argumentos
9     entero resultado = sumar(num1, num2);
10    escribir("El resultado de la suma es: ");
11    escribir(resultado);
12 }
13
14 // Función sin retorno (vacío)
15 vacio saludar() {
16     escribir("Hola, este es un ejemplo de función.");
17 }
18
19 // Función con retorno (entero) y argumentos
20 entero sumar(entero a, entero b) {
21     entero suma = a + b;
22     retornar suma;
23 }
24
25 }
```

Características del lenguaje diseñado

Estructuras de control

3 tipos:

- **Condicional:** Esta se hará con la estructura → **si** (condición) {bloque de código}. Opcionalmente, se podrá añadir una condición “contraria” con la estructura → **sino** {bloque de código}.
- **Bucle Mientras:** Esta se hará con la estructura → **mientras** (condición) {bloque de código} hasta que se deje de cumplir la condición puesta.
- **Bucle Para:** Esta se hará con la estructura → **para** (variable; condición; incremento) {bloque de código}.

```
1  vacio main() {  
2      // Condicional  
3      entero a = 10;  
4      si (a > 5) {  
5          escribir("a es mayor que 5");  
6      } sino {  
7          escribir("a no es mayor que 5");  
8      }  
9  
10     // Bucle Mientras  
11     entero contador = 0;  
12     mientras (contador < 3) {  
13         escribir("Contador: ");  
14         contador = contador + 1;  
15         escribir(contador);  
16     }  
17     // Bucle Para  
18     para (entero i = 0; i < 5; i++) {  
19         escribir("Iteración: ");  
20         escribir(i);  
21     }  
22 }
```

Características del lenguaje diseñado

Operadores

3 grupos:

- **Relacionales:** Son aquellos que nos permiten comparar valores → (=, ==, !=, <, >, <=, >=).
- **Lógicos:** Son aquellos que nos permiten realizar las operaciones lógicas más básicas → [&& (AND), || (OR), ! (NOT)].
- **Aritméticos:** Son los típicos: Suma (+), resta (-), multiplicación (*) y división (/).

```
1  vacio main() {  
2      // --- Operadores Relacionales ---  
3      entero a = 10;  
4      entero b = 5;  
5      escribir("a == b es:");  
6      escribir(a == b);           // Igualdad  
7      escribir("a != b es:");  
8      escribir(a != b);           // Desigualdad  
9      escribir("a > b es:");  
10     escribir(a > b);             // Mayor que  
11     escribir("a < b es:");  
12     escribir(a < b);             // Menor que  
13     escribir("a >= b es:");  
14     escribir(a >= b);            // Mayor o igual  
15     escribir("a <= b es:");  
16     escribir(a <= b);            // Menor o igual  
17  
18     // --- Operadores Lógicos ---  
19     booleano x = verdadero;  
20     booleano y = falso;  
21     escribir("x && y (AND) es:");  
22     escribir(x && y);             // Y lógico  
23     escribir("x || y (OR) es:");  
24     escribir(x || y);            // O lógico  
25     escribir("!x (NOT) es:");  
26     escribir(!x);                // Negación  
27 }
```

Gestión de Errores

¿Qué errores gestionamos y cómo?

¿Qué errores gestiona?

Errores Léxicos - Problemas en el análisis de caracteres y símbolos

- Caracteres inválidos o no reconocidos

Errores Sintácticos - Problemas en la estructura del programa

- Tokens en orden incorrecto
- Falta de palabras clave o símbolos esperados

Errores Semánticos - Problemas lógicos del programa

- Incompatibilidad de tipos en asignaciones
- Operaciones entre tipos no permitidos
- Variables no declaradas o redeclaradas
- Incompatibilidad de tipos en operaciones aritméticas, relacionales y lógicas
- Parámetros incorrectos en llamadas de función
- Return en función sin tipo o con tipo incorrecto

¿Cómo lo hace?

Arquitectura:

- Clase GestorErrores centraliza la gestión
- Cada error almacena: línea, tipo y mensaje descriptivo

Proceso:

- Registro - Errores se registran en una lista durante compilación
- Notificación - Se imprime en consola inmediatamente
- Reporte final - Se genera archivo errores.txt con informe completo
- Validación - Flag hayErrores detiene generación de código si hay fallos

Características:

- Reporta número total de errores
- Especifica línea exacta del error
- Compilación se detiene si hay errores

Semántico

ComprobadorTipos

Esta clase actúa como el validador de reglas lógicas del compilador, asegurando que las operaciones y asignaciones sean coherentes antes de generar el código final.

1. Funciones Principales

- **Validación de Operaciones:** Verifica que los operadores (aritméticos, lógicos y relacionales) se usen con tipos compatibles.
- **Control de Asignaciones:** Asegura que el valor asignado a una variable coincida estrictamente con su tipo declarado.
- **Integridad de Funciones:** Comprueba que el número y tipo de **argumentos** en una llamada sean correctos y valida que el valor de **retorno** coincida con el tipo definido de la función.

2. Lógica de Comprobación

- **Verificación de Condiciones:** Restringe el uso de sentencias de control (si/mientras) únicamente a tipos booleano o entero.
- **Criterio de Compatibilidad:** Define que dos tipos son compatibles solo si son idénticos o si no representan un estado de error previo.
- **Manejo de Errores en Cascada:** Ante una operación inválida, devuelve un **TipoDato.ERROR** para marcar el fallo sin detener el análisis del resto del código.

Semántico

TipoDato

Esta clase enumera los tipos permitidos por el lenguaje.

Clasificación de Tipos Soportados

- **Tipos Estándar:** entero, carácter, booleano y cadena. Representan los valores básicos con los que se puede programar.
- **Tipos de Control y Gestión:** **void:** Utilizado para funciones que no retornan ningún valor. **error:** Estado especial para marcar tipos inválidos tras un fallo semántico.
- **Tipo Estructural:** **array:** Identifica colecciones de datos del mismo tipo.
- **Conversión a Texto:** Es un método que traduce las constantes internas a cadenas legibles para generar mensajes de error claros al usuario.

Semántico

TablaSimbolos

almacena información sobre todas las variables, funciones y constantes del programa. Permite verificar redeclaraciones, respetar ámbitos y validar el uso de identificadores.

Componentes Principales

Pila de Ámbitos: Implementa ámbitos anidados mediante una pila de tablas hash, permitiendo:

- Ámbito global (nivel 0)
- Ámbitos de funciones
- Ámbitos de bloques anidados (si/mientras/para)

Clase Símbolo: Estructura que almacena:

- Nombre, tipo de dato, línea de declaración
- Ámbito donde se declaró
- Si es parámetro, constante o array
- Dimensiones (para arrays multidimensionales)
- Lista de parámetros (para funciones)
- Valor (para constantes)

Funcionalidades Clave

Gestión de Ámbitos: Métodos entrarAmbito() y salirAmbito() para manejar bloques anidados.

Búsqueda Jerárquica:

- buscar(): Busca en todos los ámbitos desde el más interno al externo
- buscarEnAmbitoActual(): Solo en el ámbito actual
- buscarGlobal(): Solo en el ámbito global

Inserción Controlada:

- insertar(): Variables simples
- insertarArray() / insertarArrayMultidimensional()
- insertarConstante(): Constantes con valor

Detección de Redeclaraciones: Verifica si un nombre ya existe en el ámbito actual.

Generación de Reportes: Método volcarAArchivo() exporta la tabla a un archivo de texto.

Backend

Código de tres direcciones

Cada vez que se genera una instrucción de tres direcciones, esta se construye como una cadena de texto y se inserta secuencialmente al final de nuestra lista de código intermedio.

Además, disponemos de métodos especializados para la gestión de recursos que generan **identificadores únicos** (temporales y etiquetas) mediante **contadores internos**, garantizando así la unicidad de las referencias para los cálculos intermedios y el control de flujo.

GeneradorCodigo
- codigoIntermedio : List<String> - contadorTemporales : int - contadorEtiquetas : int - cadenasLiterales : Map<String, String> - contadorCadenas : int
+ GeneradorCodigo() + nuevoTemporal() : String + nuevaEtiqueta() : String + registrarCadena(cadena : String) : String + emitir(instruccion : String) : void + emitirOperacion(op1 : String, operador : String, op2 : String) : String + emitirOperacionUnaria(operador : String, operando : String) : String + emitirAsignacion(destino : String, fuente : String) : void + emitirEtiqueta(etiqueta : String) : void + emitirSalto(etiqueta : String) : void + emitirSaltoCondicional(condicion : String, etiqueta : String) : void + emitirSaltoCondicionalFalso(condicion : String, etiqueta : String) : void + emitirLlamada(funcion : String, numParametros : int) : void + emitirRetorno(valor : String) : void + emitirParametro(param : String) : void + volcarAArchivo(rutaArchivo : String) : void + imprimir() : void + getInstrucciones() : List<String> + getCadenasLiterales() : Map<String, String>

Backend

Optimizador

Las optimizaciones se aplican de manera secuencial, comenzando por:

- La **eliminación de código muerto**, que detecta y suprime instrucciones inalcanzables situadas tras saltos incondicionales mediante un algoritmo de dos pasadas.
- A continuación, se ejecuta la **propagación de constantes**, que sustituye las referencias a variables por sus valores directos utilizando un mapa de seguimiento que distingue entre ámbitos locales y globales.
- Finalmente, se realiza la **eliminación de subexpresiones comunes**, la cual evita cálculos redundantes reutilizando resultados previos almacenados en variables temporales, logrando así un código final más limpio y eficiente.

Backend

Ensamblador

Realiza la transformación final del código intermedio a instrucciones de bajo nivel.

Objetivos Principales

Traducción a MIPS: Convierte cada instrucción de tres direcciones a su equivalente en instrucciones MIPS (Microprocessor without Interlocked Pipelined Stages). Ejecutable en simuladores como MARS, que ha sido el elegido para nuestro proyecto

Gestión de Memoria: Asigna localizaciones en memoria para:

- Variables globales (datos estáticos)
- Variables locales (stack)
- Arrays (en heap o data segment)

Gestión de Registros: Utiliza registros MIPS (\$a0-\$a3, \$v0-\$v1, \$t0-\$t9) para operaciones y paso de parámetros.

Backend

Ensamblador

Componentes Clave

Prólogo del programa: Inicializa la sección .data con cadenas literales y variables globales.

Mapeo de variables: Mantiene mapa entre identificadores del código intermedio y direcciones de memoria.

Manejo de funciones:

- Gestión de parámetros (stack y registros)
- Control de pila (frame pointer)
- Llamadas a funciones (jal/jr)

Operaciones especiales:

- Entrada/salida (lectura y escritura)
- Acceso a arrays
- Manejo de cadenas y caracteres

Características

Prepass de análisis: Detecta variables globales, arrays y constantes antes de la traducción.

Generación secuencial: Traduce instrucción por instrucción respetando el orden.

Epílogo del programa: Finaliza con instrucción de salida.

Archivo de salida: La optimización se hace a nivel de código intermedio lo cual genera dos versiones de este. Luego ambas versiones se traducen a MIPS. Genera codigo_sin_optimizar.asm y codigo_optimizado.asm con el código MIPS listo para ejecutar.